
Phân tích và xử lý dữ liệu lớn dựa trên dữ liệu sàn chứng khoán Việt Nam

Nguyễn Minh Quân

VNU University of Engineering and Technology

23020417@vnu.edu.vn

Github project: <https://github.com/sleepifox/stockvn-bigdata>

Tóm tắt

Báo cáo này tóm lược dự án phân tích dữ liệu lớn trên sàn chứng khoán Việt Nam, với mục tiêu so sánh các mã cổ phiếu lớn và xây dựng mô hình dự báo giá. Sử dụng hệ sinh thái Big Data gồm HDFS, PySpark, cùng các thư viện Pandas và Matplotlib, dự án đã phân tích dữ liệu của 5 công ty (VIC, VCB, FPT, HPG, GAS) để so sánh biến động và đánh giá độ bất ổn. Nổi bật là việc xây dựng mô hình học sâu LSTM để dự báo giá cổ phiếu VIC, cho thấy khả năng dự đoán xu hướng tốt. Dự án này minh chứng tiềm năng của việc ứng dụng công nghệ dữ liệu lớn và học máy trong lĩnh vực tài chính.

1 Giới thiệu

Dữ liệu giao dịch chứng khoán Việt Nam, với khối lượng lớn và tính phức tạp, là nguồn tài nguyên giá trị cần các công cụ phân tích hiện đại. Báo cáo này được thực hiện nhằm mục tiêu:

- Sử dụng Apache Hadoop (HDFS) và Spark (PySpark) để xử lý hiệu quả dữ liệu chứng khoán lớn.
- Phân tích và trực quan hóa xu hướng giá và độ biến động của các mã cổ phiếu chính.
- Xây dựng mô hình học sâu LSTM để dự báo giá cổ phiếu, kiểm chứng ứng dụng AI.

Ý nghĩa của báo cáo nằm ở việc cung cấp thông tin phân tích thị trường dựa trên dữ liệu thực tế và minh chứng khả năng áp dụng các công nghệ Big Data, Học máy vào lĩnh vực tài chính tại Việt Nam, mang lại giá trị tham khảo cho nhà đầu tư và nghiên cứu khoa học.

2 Dữ liệu chứng khoán Việt Nam

2.1 Nguồn và phạm vi dữ liệu

Trong báo cáo này, dữ liệu được thu thập từ thị trường chứng khoán Việt Nam thông qua API vnstock, một công cụ mã nguồn mở hỗ trợ truy cập thông tin giá và khối lượng giao dịch từ các sàn HOSE, HNX và UPCoM. Tổng cộng có hơn 1600 mã cổ phiếu được hỗ trợ, bao gồm nhiều doanh nghiệp quy mô lớn và có tính thanh khoản cao như VIC, VNM, VCB, HPG, SSI và CTI.

Khác với các bộ dữ liệu quốc tế thường được sử dụng trong các nghiên cứu trước, dữ liệu từ vnstock phản ánh đặc trưng vận động của thị trường trong nước, nơi biên độ dao động giá và tần suất giao dịch có tính đặc thù riêng. Việc sử dụng dữ liệu nội địa giúp mô hình có tính thực tiễn cao hơn và phù hợp với bối cảnh Việt Nam.

Ví dụ đoạn mã Python để truy xuất dữ liệu lịch sử cho mã cổ phiếu VCI:

```
from vnstock import Quote

vci = Quote('VCI', source='VCI')
data_vci = vci.history(start='2020-01-01', end='2024-05-25')
data_vci.to_csv('stockvn/VCI.csv')
```

2.2 Tiền xử lý và tổ chức dữ liệu

Sau khi tải về, mỗi mã cổ phiếu được lưu thành một tệp CSV riêng biệt trong thư mục /stockvn/. Mỗi tệp chứa các trường thông tin cơ bản:

- **Date:** ngày giao dịch.
- **Open, High, Low, Close:** các mức giá trong ngày.
- **Volume:** tổng khối lượng giao dịch.

Các tệp dữ liệu được rà soát để loại bỏ các dòng trùng lặp, chuẩn hóa định dạng thời gian và sắp xếp lại theo thứ tự tăng dần của ngày. Sau đó, dữ liệu được gộp hoặc chọn lọc tùy theo yêu cầu của từng bài toán phân tích (ví dụ: chỉ giữ lại nhóm VN30 hoặc các mã có khối lượng giao dịch cao nhất).

2.3 Đưa dữ liệu lên hệ thống HDFS

Để phục vụ cho các bài toán xử lý dữ liệu lớn, toàn bộ các tệp CSV được nạp lên hệ thống lưu trữ phân tán HDFS. Việc này giúp đảm bảo khả năng mở rộng và tính song song khi phân tích bằng Spark.

Các lệnh thực thi cụ thể:

```
hdfs dfs -mkdir -p /user/root/stockvn/
hdfs dfs -put stockvn /user/root/
```

Sau khi tải lên, ta có thể kiểm tra bằng:

```
hdfs dfs -ls /user/root/stockvn/
```

Nếu các tệp hiển thị đầy đủ, điều đó xác nhận dữ liệu đã được lưu trữ thành công trên HDFS và sẵn sàng cho các bước xử lý tiếp theo bằng Spark hoặc PySpark.stockvn

2.4 Tóm tắt quy trình

Toàn bộ quy trình xử lý dữ liệu được thực hiện theo các bước sau:

1. Thu thập dữ liệu lịch sử cổ phiếu từ API vnstock.
2. Lưu dữ liệu từng mã dưới dạng CSV trong thư mục /stockvn/.
3. Làm sạch và chuẩn hóa dữ liệu đầu vào.
4. Tải dữ liệu lên HDFS để phục vụ xử lý phân tán.

3 Xử lý và trực quan hoá dữ liệu

Quá trình xử lý và trực quan hoá được thực hiện trong môi trường Spark nhằm tận dụng khả năng phân tán dữ liệu, sau đó chuyển sang Pandas để vẽ biểu đồ và phân tích chi tiết. Các thao tác chính bao gồm kết nối Spark, xử lý dữ liệu, tính toán đặc trưng, và trực quan hoá kết quả.

3.1 Kết nối và đọc dữ liệu từ HDFS

Đầu tiên, chương trình thiết lập phiên làm việc với Spark thông qua địa chỉ master của cụm, đồng thời đọc các tệp dữ liệu chứng khoán từ hệ thống HDFS. Mỗi tệp tương ứng với một mã cổ phiếu cụ thể.

```
spark = SparkSession.builder.master("spark://172.18.0.2:7077")\
    .appName("Stock_price_analysis").getOrCreate()

df_VIC = spark.read.format("csv").option("header", "true")\
    .option("inferSchema", "true").load("hdfs://namenode:9000/user/\
    ↪ root/stockvn/VIC.csv")

df_VCB = spark.read.format("csv").option("header", "true")\
    .option("inferSchema", "true").load("hdfs://namenode:9000/user/\
    ↪ root/stockvn/VCB.csv")

df_FPT = spark.read.format("csv").option("header", "true")\
    .option("inferSchema", "true").load("hdfs://namenode:9000/user/\
    ↪ root/stockvn/FPT.csv")

df_HPG = spark.read.format("csv").option("header", "true")\
    .option("inferSchema", "true").load("hdfs://namenode:9000/user/\
    ↪ root/stockvn/HPG.csv")

df_GAS = spark.read.format("csv").option("header", "true")\
    .option("inferSchema", "true").load("hdfs://namenode:9000/user/\
    ↪ root/stockvn/GAS.csv")
```

3.2 Tính toán giá trung bình

Mỗi mã cổ phiếu có các cột high và low thể hiện giá cao nhất và thấp nhất trong ngày. Để phản ánh xu hướng giá trung tâm, giá trung bình được tính bằng công thức:

$$mean = \frac{high + low}{2}$$

```
from pyspark.sql.functions import expr

df_VICmean = df_VIC.withColumn("mean", expr("(high+low)/2"))
df_VCBmean = df_VCB.withColumn("mean", expr("(high+low)/2"))
df_FPTmean = df_FPT.withColumn("mean", expr("(high+low)/2"))
df_HPGmean = df_HPG.withColumn("mean", expr("(high+low)/2"))
df_GASmean = df_GAS.withColumn("mean", expr("(high+low)/2"))
```

3.3 Chuyển dữ liệu sang Pandas để trực quan hoá

Sau khi xử lý trên Spark, dữ liệu được chuyển sang định dạng Pandas DataFrame để thuận tiện cho việc trực quan hoá bằng matplotlib.

```
df_VICmean = df_VICmean.toPandas()
df_VCBmean = df_VCBmean.toPandas()
df_FPTmean = df_FPTmean.toPandas()
df_HPGmean = df_HPGmean.toPandas()
df_GASmean = df_GASmean.toPandas()
```

3.4 Trực quan hoá xu hướng giá trung bình

Biểu đồ đường thể hiện sự thay đổi của giá trung bình qua thời gian giúp so sánh xu hướng biến động giữa các mã cổ phiếu lớn như VIC, VCB, FPT, HPG và GAS.

```

ax = df_VICmean.plot(x='time', y='mean', label='VIC', figsize=(16,6))
ax = df_VCBmean.plot(ax=ax, x='time', y='mean', label='VCB')
ax = df_HPGmean.plot(ax=ax, x='time', y='mean', label='HPG')
ax = df_GASmean.plot(ax=ax, x='time', y='mean', label='GAS')
df_FPTmean.plot(ax=ax, x='time', y='mean', label='FPT')

```

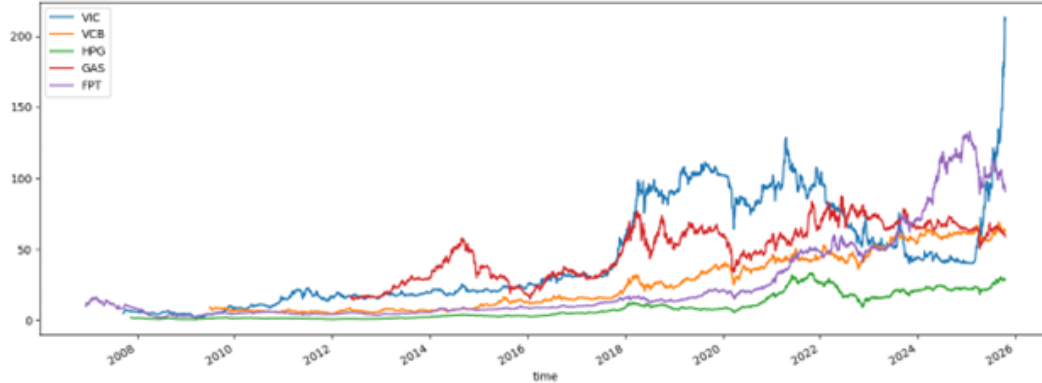


Figure 1: Biểu đồ xu hướng giá trung bình

Biểu đồ này cho thấy xu hướng chung của nhóm cổ phiếu đầu ngành, đồng thời thể hiện rõ thời điểm biến động mạnh của từng mã.

3.5 Phân tích biến động tỷ suất sinh lời

Để đánh giá rủi ro và phân phối lợi nhuận, ta tính tỷ suất thay đổi hàng ngày:

$$change_t = \frac{Close_t}{Close_{t-1}} - 1$$

```

df_VICmean['change'] = (df_VICmean['close']/df_VICmean['close'].shift
↳ (1)) - 1
df_VCBmean['change'] = (df_VCBmean['close']/df_VCBmean['close'].shift
↳ (1)) - 1
df_GASmean['change'] = (df_GASmean['close']/df_GASmean['close'].shift
↳ (1)) - 1

```

Sau đó, biểu đồ histogram được vẽ để quan sát phân phối của biến động giá, cho thấy phần lớn biến động tập trung quanh giá trị trung bình gần 0.

```

df_VICmean['change'].hist(bins=100, label='VIC', alpha=0.5)
df_VCBmean['change'].hist(bins=100, label='VCB', alpha=0.5)
df_GASmean['change'].hist(bins=100, label='GAS', alpha=0.5)
plt.legend()

```

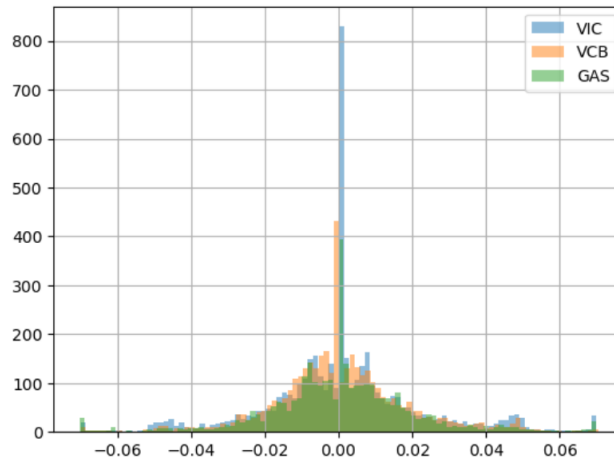


Figure 2: Biểu đồ phân phối biến động giá

3.6 Phân tích mật độ xác suất (KDE)

Cuối cùng, biểu đồ mật độ (KDE) giúp trực quan hoá dạng phân phối của lợi nhuận, qua đó xác định mức độ biến động của từng mã cổ phiếu.

```
df_VICmean['change'].plot(kind='kde', label='VIC')
df_VCBmean['change'].plot(kind='kde', label='VCB')
df_GASmean['change'].plot(kind='kde', label='GAS')
plt.legend()
```

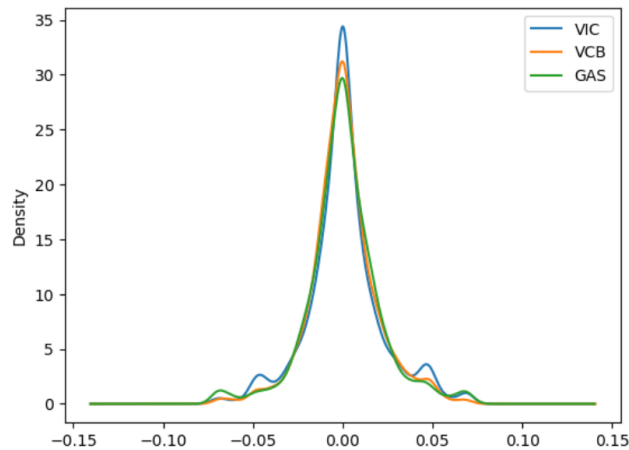


Figure 3: Đường KDE mật độ xác suất

Các đường KDE cho thấy độ rộng và độ lệch khác nhau giữa các mã, phản ánh rủi ro thị trường riêng biệt của từng doanh nghiệp.

3.7 Nhận xét các biểu đồ trực quan hóa dữ liệu

3.7.1 Biểu đồ 1: Diễn biến giá cổ phiếu theo thời gian

Biểu đồ đường thể hiện xu hướng giá cổ phiếu của năm mã VIC, VCB, HPG, GAS và FPT trong giai đoạn từ năm 2008 đến năm 2025.

Từ biểu đồ có thể nhận thấy rằng hầu hết các cổ phiếu đều có xu hướng tăng trưởng dài hạn, mặc dù có những giai đoạn biến động mạnh (đặc biệt trong giai đoạn 2020–2022 do ảnh hưởng của dịch COVID-19 và biến động kinh tế toàn cầu).

Cụ thể:

- **VIC** có biên độ dao động mạnh nhất, đạt đỉnh gần 200, thể hiện sự tăng trưởng nhanh nhưng kèm theo rủi ro cao.
- **VCB** và **FPT** tăng ổn định hơn, thể hiện khả năng tăng trưởng bền vững trong dài hạn.
- **GAS** có xu hướng giảm mạnh ở một số giai đoạn, phản ánh sự phụ thuộc vào biến động giá năng lượng toàn cầu.

Biểu đồ cho thấy sự khác biệt rõ rệt giữa các cổ phiếu về tốc độ tăng trưởng và mức độ rủi ro, giúp nhà đầu tư đánh giá được tiềm năng và chiến lược phù hợp cho từng mã.

3.7.2 Biểu đồ 2: Phân bố lợi nhuận hàng ngày (Histogram)

Biểu đồ histogram mô tả phân bố tần suất của lợi nhuận hàng ngày đối với ba cổ phiếu VIC, VCB và GAS.

Phân bố có dạng chuông, tập trung chủ yếu quanh giá trị trung tâm là 0, cho thấy đa số các ngày giao dịch có mức biến động nhỏ. Tuy nhiên, phần đuôi của phân bố kéo dài về hai phía âm và dương, biểu hiện sự tồn tại của các ngày biến động mạnh, tức là có các cú sốc giá đột ngột.

- **VIC** có đỉnh tần suất cao nhất tại 0, thể hiện giá thường xuyên ổn định, nhưng vẫn có biên độ biến động rộng hơn so với các mã còn lại.
- **VCB** và **GAS** có phân bố hẹp hơn, phản ánh mức độ rủi ro thấp hơn.

Biểu đồ histogram cung cấp góc nhìn định lượng về rủi ro và tần suất xuất hiện của các mức lợi nhuận khác nhau trong quá trình giao dịch.

3.7.3 Biểu đồ 3: Mật độ xác suất (KDE Plot)

Biểu đồ KDE thể hiện đường mật độ xác suất của lợi nhuận hàng ngày đối với ba cổ phiếu VIC, VCB và GAS.

Các đường mật độ đều có đỉnh nhọn tại vị trí gần 0, chứng tỏ phần lớn lợi nhuận hàng ngày dao động xung quanh mức trung bình, và phần lớn biến động là nhỏ. Độ nhọn của các đường cao cho thấy sự tập trung lớn quanh giá trị trung bình, nhưng vẫn tồn tại các ngoại lệ ở hai đầu (outliers).

- **VIC** có đường phân bố rộng hơn, thể hiện biến động mạnh và rủi ro cao hơn.
- **VCB** và **GAS** có đường phân bố hẹp, phản ánh mức ổn định cao hơn.

So với biểu đồ histogram, KDE plot cho phép quan sát phân bố một cách mượt mà và trực quan hơn, giúp so sánh sự ổn định giữa các cổ phiếu.

3.7.4 Tổng kết

Tổng hợp lại, ta nhận thấy VIC có mức biến động mạnh nhất, trong khi VCB và FPT có xu hướng ổn định hơn, phù hợp với chiến lược đầu tư dài hạn. GAS thể hiện tính chu kỳ rõ rệt do ảnh hưởng của ngành năng lượng.

4 Mô hình dự báo giá chứng khoán sử dụng LSTM

Để giải quyết bài toán dự báo chuỗi thời gian như giá cổ phiếu, mô hình **Hồi quy Tuyến tính** hay các mô hình học máy truyền thống thường không hiệu quả vì chúng không có khả năng "ghi nhớ" các sự kiện trong quá khứ. Mạng **LSTM (Long Short-Term Memory)**, một dạng đặc biệt của mạng nơ-ron hồi quy (RNN), ra đời để khắc phục nhược điểm này. LSTM có khả năng học và ghi nhớ các phụ thuộc dài hạn, làm cho nó trở thành một công cụ cực kỳ mạnh mẽ để phân tích và dự báo dữ liệu chuỗi thời gian.

Quy trình xây dựng mô hình được chia thành các bước chính sau:

4.1 Tiền xử lý và Chuẩn bị dữ liệu

Đây là bước quan trọng nhất, quyết định phần lớn hiệu quả của mô hình. Dữ liệu thô cần được xử lý và chuyển đổi thành định dạng phù hợp cho mạng LSTM.

4.1.1 Tải dữ liệu và tạo đặc trưng

Đầu tiên, chúng ta sử dụng **PySpark** để đọc dữ liệu giá cổ phiếu từ HDFS và tạo ra một cột mean bằng trung bình cộng của giá cao nhất (high) và giá thấp nhất (low) để làm đặc trưng chính.

```
from pyspark.sql.functions import expr

df = spark.read.format("csv")\
    .option("header", "true")\
    .option("inferSchema", "true")\
    .load("hdfs://namenode:9000/user/root/stockvn/VIC.csv")

df = df.withColumn("mean", expr('(high+low)/2'))
```

4.1.2 Phân chia dữ liệu Train-Test

Đối với dữ liệu chuỗi thời gian, việc phân chia phải tuân theo thứ tự thời gian. Chúng ta lấy dữ liệu từ năm 2024 trở về trước để huấn luyện và dữ liệu của năm 2025 để kiểm thử.

```
asc_df = df.sort(df.time.asc())
df_test = asc_df.filter(year("time") > 2024)
df_train = asc_df.subtract(df_test)
```

4.1.3 Chuẩn hóa dữ liệu (Scaling)

Các mạng nơ-ron hoạt động tốt nhất khi giá trị đầu vào nằm trong khoảng từ 0 đến 1. Chúng ta sử dụng **MinMaxScaler** để chuẩn hóa dữ liệu. **Lưu ý quan trọng:** Scaler này sẽ được dùng lại để biến đổi tập test.

```
dataset_train = df_train.toPandas()
training_set = dataset_train.iloc[:, 6:7].values

sc = MinMaxScaler(feature_range = (0, 1))
training_set_scaled = sc.fit_transform(training_set)
```

4.1.4 Tạo chuỗi dữ liệu (Windowing)

Đây là bước cốt lõi để chuẩn bị dữ liệu cho LSTM. Mô hình sẽ học cách dự đoán giá của ngày tiếp theo dựa trên giá của **60 ngày** trước đó. Vòng lặp sẽ duyệt qua dữ liệu, lấy một chuỗi 60 ngày làm đầu vào (X_train) và ngày ngay sau đó làm nhãn/đầu ra (y_train). Cuối cùng, dữ liệu được reshape thành định dạng 3D mà LSTM yêu cầu.

```
X_train = []
y_train = []
no_of_sample = len(training_set_scaled)

for i in range(60, no_of_sample):
    X_train.append(training_set_scaled[i-60:i, 0])
    y_train.append(training_set_scaled[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

4.2 Xây dựng và Huấn luyện mô hình LSTM

Chúng ta xây dựng một mô hình LSTM xếp chồng (stacked LSTM) bằng thư viện Keras.

- **LSTM Layers:** Gồm 4 lớp LSTM, mỗi lớp có 50 units. Các lớp ở giữa có `return_sequences=True` để truyền chuỗi kết quả cho lớp tiếp theo.
- **Dropout Layers:** Lớp Dropout với tỷ lệ 0.2 được thêm vào để ngăn chặn hiện tượng **overfitting**.
- **Dense Layer:** Lớp đầu ra cuối cùng là một lớp Dense với 1 unit để dự đoán giá trị duy nhất.
- **Compile:** Mô hình được biên dịch với optimizer adam và hàm mất mát `mean_squared_error`.

```
regressor = Sequential()
regressor.add(LSTM(units = 50, return_sequences = True, input_shape =
    ↪ (X_train.shape[1], 1)))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units = 50, return_sequences = True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units = 50, return_sequences = True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units = 50))
regressor.add(Dropout(0.2))
regressor.add(Dense(units = 1))
```

Chúng ta sẽ huấn luyện mô hình bằng tối ưu hoá Adam cùng với hàm loss MSE, ta sẽ training trên 20 epochs và batch size là 32.

```
regressor.compile(optimizer = 'adam', loss = 'mean_squared_error')
regressor.fit(X_train, y_train, epochs = 100, batch_size = 32)
```

4.3 Dự đoán và Trực quan hoá kết quả

Sau khi huấn luyện, mô hình được dùng để dự đoán trên tập test. Quá trình chuẩn bị dữ liệu test phải tương tự như tập train, đặc biệt là phải sử dụng lại đối tượng scaler đã được fit trước đó.

```
dataset_total = pd.concat((dataset_train['mean'], dataset_test['mean',
    ↪ ]), axis = 0)
inputs = dataset_total[len(dataset_total) - len(dataset_test) - 60:].
    ↪ values
inputs = inputs.reshape(-1,1)
inputs = sc.transform(inputs)

X_test = []
for i in range(60, len(inputs)):
    X_test.append(inputs[i-60:i, 0])
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

predicted_stock_price = regressor.predict(X_test)
predicted_stock_price = sc.inverse_transform(predicted_stock_price)
```

Chúng ta sẽ thực hiện trực quan hoá kết quả bằng cách hiển thị đường giá chính xác (màu đỏ) và giá dự đoán (màu xanh) của cổ phiếu VIC.


```
import matplotlib.pyplot as plt

plt.figure(figsize=(16,9))
plt.plot(real_stock_price, color = 'red', label = 'Real VIC Stock
↪ Price')
plt.plot(predicted_stock_price, color = 'blue', label = 'Predicted VIC
↪ Stock Price')
plt.title('VIC Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('VIC Stock Price')
plt.legend()
plt.show()
```

4.4 Kết quả và nhận xét mô hình

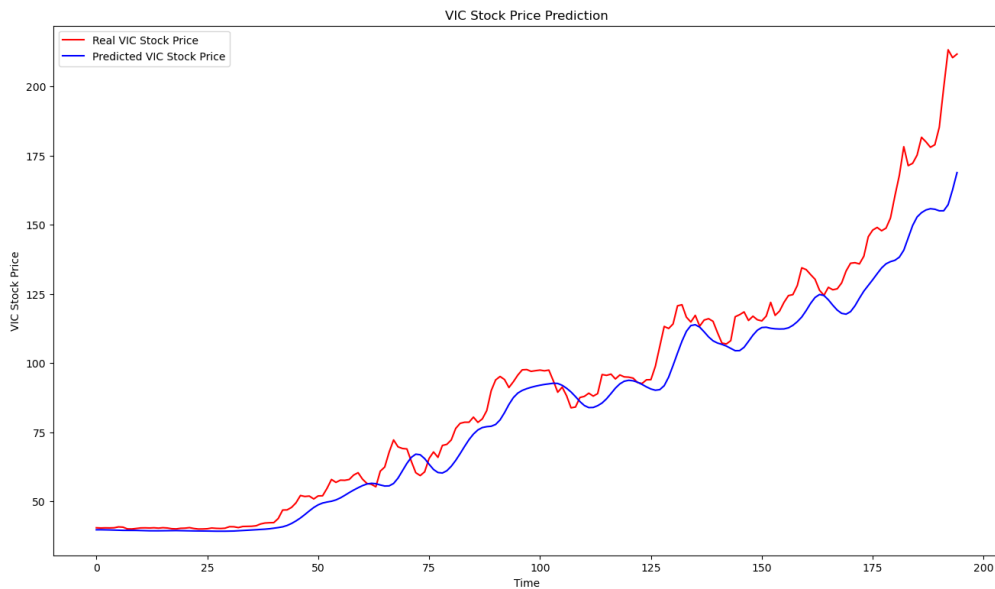


Figure 4: Biểu đồ so sánh giá thực tế và giá dự đoán cổ phiếu VIC bằng mô hình LSTM

Hình 4 thể hiện kết quả dự báo giá cổ phiếu VIC bằng mô hình LSTM. Đường màu **đỏ** biểu diễn giá thực tế, trong khi đường **xanh** biểu diễn giá dự đoán từ mô hình.

Nhận xét:

- Mô hình LSTM đã **nắm bắt khá tốt xu hướng chung** của giá cổ phiếu — khi giá tăng, mô hình cũng dự báo tăng và ngược lại.
- Tuy nhiên, **giá dự đoán vẫn có độ trễ và biên độ dao động nhỏ hơn** so với giá thực tế. Nguyên nhân có thể do:
 - Số epoch huấn luyện còn ít (20 epochs), nên mô hình chưa học sâu các mẫu biến động phức tạp.
 - Dữ liệu đầu vào chỉ gồm giá trung bình (mean) mà chưa bổ sung thêm các đặc trưng kỹ thuật như RSI, MACD, MA.
 - Mô hình có thể bị regularization mạnh (Dropout 0.2 ở mọi lớp), làm giảm khả năng fitting các biến động nhỏ.
- Nhìn chung, mô hình đã **dự báo được xu hướng chính xác**, nhưng cần tinh chỉnh thêm để giá trị dự đoán khớp sát hơn.

5 Kết luận

Dự án này đã tiến hành phân tích và dự đoán giá cổ phiếu Việt Nam thông qua nhiều bước xử lý và trực quan hóa dữ liệu. Trước hết, dữ liệu giá cổ phiếu được thu thập, làm sạch và chuẩn hóa nhằm loại bỏ các giá trị thiếu và sai lệch. Sau đó, quá trình trực quan hóa được thực hiện để khám phá đặc điểm của dữ liệu qua các biểu đồ như biểu đồ thời gian, histogram và đồ thị mật độ (KDE). Các biểu đồ này giúp nhận diện xu hướng tăng giảm, biến động của giá cổ phiếu theo thời gian, cũng như phân bố tần suất và mức độ tập trung của dữ liệu.

Tiếp theo, mô hình học máy được triển khai nhằm dự đoán xu hướng giá cổ phiếu trong tương lai. Mô hình được huấn luyện dựa trên dữ liệu lịch sử, qua đó có khả năng ước lượng giá trị tiếp theo dựa trên xu hướng trước đó. Kết quả dự đoán (Hình 4) cho thấy mô hình có thể phản ánh tương đối chính xác xu hướng biến động của giá, đặc biệt trong các giai đoạn ổn định. Tuy nhiên, ở những thời điểm thị trường có biến động mạnh, độ sai lệch giữa giá thực tế và giá dự đoán vẫn còn đáng kể.

Nhìn chung, kết quả cho thấy tiềm năng ứng dụng của các phương pháp phân tích dữ liệu và học máy trong việc hỗ trợ ra quyết định đầu tư. Tuy nhiên, để mô hình đạt độ chính xác cao hơn, có thể mở rộng theo các hướng sau:

- Bổ sung thêm các biến đầu vào như khối lượng giao dịch, dữ liệu kinh tế vĩ mô, hoặc chỉ số thị trường.
- Tối ưu hóa mô hình bằng các thuật toán hiện đại như Transformer.
- Tăng cường khả năng tổng quát bằng cách huấn luyện trên dữ liệu nhiều năm hoặc nhiều mã cổ phiếu khác nhau.

6 Tài liệu tham khảo

- API vnstock - <https://vnstocks.com/docs>
- Dự án tham khảo - <https://github.com/thviet79/Stock-Price>